COMPUTER NETWORKS LAB

PRACTICAL - 1

Implement Concurrent Echo Client-Server Application in Java

Name	_____
Roll No.	_____
Class / Semester	_____
Date	_____

Aim: To implement a concurrent echo client-server application in Java using TCP sockets and multithreading.

Software Required: JDK 8 or later, terminal/command prompt, and any text editor or IDE.

1. Theory

A client-server application contains a client, a server, and a network between them. For this practical, TCP is used as the transport protocol because it is connection-oriented and provides reliable data transfer between two computers.

A **socket** is an endpoint of a two-way communication link between programs. Java provides the **Socket** class for clients and the **ServerSocket** class for servers. The server listens on a port and accepts incoming client connections.

A normal looping server can accept clients one after another. In a **concurrent server**, every accepted client is assigned to a separate thread. Therefore, multiple clients can send messages and receive replies at the same time.

Echo Operation

```
Client 1 ----> Thread 1 --\
Client 2 ----> Thread 2 ----> Concurrent Echo Server (Port 1234)
Client 3 ----> Thread 3 --/

Client sends:  Hello Server
Server replies: Hello Server
```

2. Algorithm

Server:

1. Create a ServerSocket on port 1234.
2. Continuously call accept() to wait for client connections.
3. For every accepted client, create a new ClientHandler thread.
4. Read a line sent by that client.
5. Send the same line back to the same client.
6. If the message is "exit", close that client connection.
7. Continue accepting and serving other clients.

Client:

1. Create a Socket connected to localhost on port 1234.
2. Create input and output streams.
3. Read a message from the keyboard.
4. Send it to the server.
5. Read and display the echoed response.
6. Repeat until the user enters "exit".
7. Close the socket.

3. Program - EchoServer.java

The server accepts connections continuously. Each connected client is handled by a separate thread through the ClientHandler class.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.ServerSocket;
import java.net.Socket;

public class EchoServer {
    private static final int PORT = 1234;

    public static void main(String[] args) {
        System.out.println("Starting Concurrent Echo Server...");

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Server is running on port " + PORT);
            System.out.println("Waiting for clients...");

            while (true) {
                Socket clientSocket = serverSocket.accept();
                System.out.println("Client connected: " + clientSocket.getRemoteSocketAddress());

                Thread clientThread = new Thread(new ClientHandler(clientSocket));
                clientThread.start();
            }
        } catch (IOException e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }
}

class ClientHandler implements Runnable {
    private final Socket clientSocket;

    ClientHandler(Socket clientSocket) {
        this.clientSocket = clientSocket;
    }

    @Override
    public void run() {
        try (
            Socket socket = clientSocket;
            BufferedReader input = new BufferedReader(
                new InputStreamReader(socket.getInputStream())
            );
            PrintWriter output = new PrintWriter(socket.getOutputStream(), true)
        ) {
            String message;

            while ((message = input.readLine()) != null) {
                System.out.println(
                    "[" + Thread.currentThread().getName() + "] " +
                    socket.getRemoteSocketAddress() + " -> " + message
                );

                // Echo the received message back to the same client.
                output.println(message);

                if (message.equalsIgnoreCase("exit")) {
                    break;
                }
            }
        } catch (IOException e) {
            System.out.println("Client communication error: " + e.getMessage());
        } finally {
            System.out.println("Client disconnected: " + clientSocket.getRemoteSocketAddress());
        }
    }
}
```

4. Program - EchoClient.java

The client connects to the server, sends messages entered by the user, and prints the same messages returned by the echo server.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.io.PrintWriter;
import java.net.Socket;
import java.util.Scanner;

public class EchoClient {
    private static final String SERVER_HOST = "localhost";
    private static final int SERVER_PORT = 1234;

    public static void main(String[] args) {
        try (
            Socket socket = new Socket(SERVER_HOST, SERVER_PORT);
            BufferedReader input = new BufferedReader(
                new InputStreamReader(socket.getInputStream())
            );
            PrintWriter output = new PrintWriter(socket.getOutputStream(), true);
            Scanner scanner = new Scanner(System.in)
        ) {
            System.out.println("Connected to Echo Server at " + SERVER_HOST + ":" + SERVER_PORT);
            System.out.println("Type 'exit' to close the client.\n");

            while (true) {
                System.out.print("Enter message: ");
                String message = scanner.nextLine();

                output.println(message);

                String response = input.readLine();
                if (response == null) {
                    System.out.println("Server closed the connection.");
                    break;
                }

                System.out.println("Server echoed: " + response);

                if (message.equalsIgnoreCase("exit")) {
                    break;
                }
            }
        } catch (IOException e) {
            System.out.println("Client error: " + e.getMessage());
        }
    }
}
```

5. Compilation and Execution

Open a terminal in the **src** directory.

Compile:

```
javac EchoServer.java EchoClient.java
```

Start the server in Terminal 1:

```
java EchoServer
```

Start a client in Terminal 2:

```
java EchoClient
```

Open Terminal 3, Terminal 4, etc. and run **java EchoClient** again to demonstrate multiple clients being served concurrently.

6. Sample Output

Server:

```
Starting Concurrent Echo Server...
Server is running on port 1234
Waiting for clients...
Client connected: /127.0.0.1:53101
Client connected: /127.0.0.1:53102
[Thread-0] /127.0.0.1:53101 -> Hello from Client 1
[Thread-1] /127.0.0.1:53102 -> Hello from Client 2
```

Client:

```
Connected to Echo Server at localhost:1234
Type 'exit' to close the client.

Enter message: Hello Server
Server echoed: Hello Server
Enter message: exit
Server echoed: exit
```

7. Result

The concurrent echo client-server application was implemented successfully using Java TCP sockets and multithreading. The server can handle multiple clients simultaneously, and each client receives the same message that it sends to the server.

8. Short Viva Questions

Q1.	**What is a socket?**	An endpoint of a two-way communication link between programs on a network.
Q2.	**Which Java classes are used here?**	ServerSocket for the server and Socket for each client connection.
Q3.	**Why is TCP used?**	TCP is connection-oriented and provides reliable data transfer.
Q4.	**Why is this server concurrent?**	Each connected client is handled by a separate thread.
Q5.	**What does accept() do?**	It waits for and accepts an incoming client connection.
Q6.	**What is an echo server?**	A server that sends received data back to the sender.

Reference: Network Programming and Java Sockets laboratory material supplied with the assignment.